

Suite de Productividad con Reproductor Musical Interactivo usando Electron + Desktop APIs.

Proyecto: *“DeskForge + BeatSync - Donde el código y la música se sincronizan”.*

Tecnologías: Electron, Node.js, HTML/CSS/JS (o framework frontend a elección), Howler.js, Web Audio API, Spotify Web API / Last.fm API, Electron IPC, Vite.

Objetivo general: Construir una aplicación de escritorio multiplataforma que combine herramientas de productividad (bloc de notas, temporizador Pomodoro, terminal rápido) con un reproductor musical interactivo con visualizaciones de audio e integración con APIs externas.

Introducción – El poder del software de escritorio.

¿Por qué Electron?

Vivimos en la era de las apps web, pero el software de escritorio no ha muerto. De hecho, algunas de las aplicaciones más usadas del mundo son de escritorio: VS Code, Slack, Discord, WhatsApp Desktop, Figma, Notion, Spotify. ¿Y saben qué tienen todas en común? Muchas están construidas con Electron.

Electron permite crear aplicaciones de escritorio multiplataforma (Windows, macOS, Linux) usando las tecnologías web que ya conocen: HTML, CSS y JavaScript. Por debajo, combina Chromium (para la interfaz) y Node.js (para acceder al sistema operativo).

¿Por qué una suite de productividad + música?

Porque es un proyecto real, útil y divertido. Y porque toca temas que van más allá del frontend:

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

- **Desktop APIs:** Acceso a archivos, notificaciones nativas, bandeja del sistema, atajos globales.
- **IPC (Inter-Process Communication):** Comunicación entre el proceso principal (Node) y el renderer (UI).
- **Audio avanzado:** Web Audio API y Howler.js para reproducir, analizar y visualizar audio.
- **Integración con APIs externas:** Spotify, Last.fm, scrobbling.
- **Empaquetado:** Generar instaladores `.exe`, `.dmg`, `.AppImage`.

¿Qué aprenderán?

- **Electron:** Procesos main/renderer, IPC, menús, atajos, bandeja.
- **Desktop APIs:** Diálogos nativos, notificaciones, acceso a archivos, auto-actualización.
- **Audio:** Howler.js para reproducción, Web Audio API para análisis de frecuencias.
- **Visualizaciones:** Barras, ondas, círculos animados con Canvas o SVG.
- **APIs externas:** OAuth con Spotify, last.fm para scrobbling.
- **Empaquetado y distribución:** electron-builder, firma, publicación.

Electron Fundamentals

Electron tiene dos procesos principales:

- **Main Process:** Node.js puro. Gestiona ventanas, menús, IPC, APIs nativas.
- **Renderer Process:** Es como un navegador (Chromium). Ejecuta la UI.

Estructura básica:

```
mi-app/  
├─ package.json  
├─ electron/  
│ └─ main.js      // Proceso principal  
│ └─ preload.js   // Puente seguro entre main y renderer
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

```
├─ src/           // UI (React, Vue, Svelte o vanilla)
|  └─ index.html
|  └─ index.js
└─ electron-builder.yml // Configuración de empaquetado
```

Ejemplo mínimo de `main.js`:

```
const { app, BrowserWindow, ipcMain, dialog, Notification } = require('electron');
const path = require('path');
```

```
let mainWindow;
```

```
function createWindow() {
  mainWindow = new BrowserWindow({
    width: 1200,
    height: 800,
    backgroundColor: '#0f0f1a',
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
      contextIsolation: true,
      nodeIntegration: false
    }
  });
};
```

```
mainWindow.loadFile('src/index.html');
}
```

```
app.whenReady().then(createWindow);
```

```
app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') app.quit();
});
```

Ejemplo de `preload.js` (exponer APIs seguras):

```
const { contextBridge, ipcRenderer } = require('electron');

contextBridge.exposeInMainWorld('api', {
  openFile: () => ipcRenderer.invoke('dialog:openFile'),
  saveFile: (content) => ipcRenderer.invoke('dialog:saveFile', content),
  notify: (title, body) => ipcRenderer.send('notify', { title, body }),
  selectMusicFolder: () => ipcRenderer.invoke('music:selectFolder')
});
```

IPC – Comunicación entre Main y Renderer

El renderer no puede acceder directamente a Node.js (por seguridad). Se comunica mediante IPC.

Patrón `invoke` / `handle` (promesas):

`main.js`:

```
ipcMain.handle('dialog:openFile', async () => {
  const result = await dialog.showOpenDialog({
    properties: ['openFile'],
    filters: [{ name: 'Text', extensions: ['txt', 'md'] }]
  });
  if (result.canceled) return null;
  return result.filePaths[0];
});
```

`renderer.js`:

```
const filePath = await window.api.openFile();
```

Patrón `send` / `on` (eventos unidireccionales):

```
// Main → Renderer
mainWindow.webContents.send('music:trackChanged', { title: 'Song' });

// Renderer escucha
ipcRenderer.on('music:trackChanged', (event, data) => { ... });
```

Desktop APIs Nativas

Diálogos de archivo:

```
const { dialog } = require('electron');
const result = await dialog.showSaveDialog({ defaultPath: 'nota.txt' });
```

Notificaciones nativas:

```
const { Notification } = require('electron');
new Notification({ title: 'Pomodoro', body: '¡Tiempo de descanso!' }).show();
```

Bandeja del sistema (System Tray):

```
const { Tray, Menu } = require('electron');
const tray = new Tray('icon.png');
tray.setContextMenu(Menu.buildFromTemplate([
  { label: 'Play/Pause', click: () => mainWindow.webContents.send('music:toggle') },
  { label: 'Salir', role: 'quit' }
]));
```

Atajos globales:

```
const { globalShortcut } = require('electron');
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

```
globalShortcut.register('CommandOrControl+Shift+Space', () => {  
  mainWindow.webContents.send('music:toggle');  
});
```

Lectura de archivos del sistema:

```
const fs = require('fs');  
const files = fs.readdirSync('/ruta/a/musica').filter(f => f.endsWith('.mp3'));
```

Audio con Howler.js

Howler.js es la librería de audio más popular para la web. Maneja formatos (MP3, OGG, WAV), streaming, sprites, y funciona bien en Electron.

```
import { Howl, Howler } from 'howler';
```

```
const sound = new Howl({  
  src: ['song.mp3'],  
  html5: true,  
  volume: 0.5,  
  onplay: () => console.log('Reproduciendo'),  
  onend: () => nextTrack()  
});
```

```
sound.play();  
sound.pause();  
sound.seek(30);    // Saltar a 30s  
sound.volume(0.8); // Cambiar volumen
```

Web Audio API – Visualizaciones

Web Audio API permite analizar el audio en tiempo real usando un **AnalyserNode**. Esto permite crear visualizaciones reactivas.

Configuración básica:

```
const audioContext = new AudioContext();
const analyser = audioContext.createAnalyser();
analyser.fftSize = 256;

// Conectar el audio source al analyser
const source = audioContext.createMediaElementSource(audioElement);
source.connect(analyser);
analyser.connect(audioContext.destination);

// Obtener datos de frecuencia
const dataArray = new Uint8Array(analyser.frequencyBinCount);
analyser.getByteFrequencyData(dataArray);
```

Visualización con Canvas:

```
function draw() {
  requestAnimationFrame(draw);
  analyser.getByteFrequencyData(dataArray);

  ctx.clearRect(0, 0, canvas.width, canvas.height);
  const barWidth = canvas.width / dataArray.length;

  dataArray.forEach((value, i) => {
    const height = (value / 255) * canvas.height;
    ctx.fillStyle = `hsl(${i * 3}, 100%, 60%)`;
    ctx.fillRect(i * barWidth, canvas.height - height, barWidth - 2, height);
  });
}
draw();
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

Tipos de visualización:

- **Barras de frecuencia:** Como el ejemplo anterior.
- **Onda (waveform):** Usar ``getBytesTimeDomainData()``.
- **Círculos radiales:** Mapear las frecuencias a ángulos.
- **Partículas:** Reactivas a los beats (detectar picos de energía).

Integración con Spotify Web API

Spotify API permite buscar canciones, obtener playlists y controlar reproducción. Requiere OAuth 2.0.

Flujo resumido:

1. Registrar la app en <https://developer.spotify.com/dashboard>.
2. Obtener ``client_id`` y ``client_secret``.
3. Implementar OAuth (Authorization Code flow con PKCE para apps desktop).
4. Usar ``fetch`` con el ``access_token`` para llamar a la API.

```
async function searchTrack(query, token) {  
  const res = await fetch(`https://api.spotify.com/v1/search?q=${query}&type=track`, {  
    headers: { Authorization: `Bearer ${token}` }  
  });  
  return res.json();  
}
```

Last.fm API (Scrobbling):

- Registrarse en <https://www.last.fm/api>.
- Usar ``track.scrobble`` para registrar las canciones reproducidas.

```
fetch(`https://ws.audioscrobbler.com/2.0/?method=track.scrobble&artist=${artist}&track=$
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

```
{title}&api_key=${API_KEY}&format=json`, { method: 'POST' });
```

Playlists con Drag & Drop

Usar la HTML5 Drag and Drop API (o una librería como `SortableJS`) para reordenar canciones.

```
item.addEventListener('dragstart', (e) => {  
  e.dataTransfer.setData('text/plain', item.dataset.id);  
});
```

```
list.addEventListener('drop', (e) => {  
  const id = e.dataTransfer.getData('text/plain');  
  reorderPlaylist(id, e.target.closest('li').dataset.id);  
});
```

Empaquetado con electron-builder

```
npm install --save-dev electron-builder
```

```
`electron-builder.yml`:
```

```
``yaml
```

```
appId: com.tuNombre.DeskForge
```

```
productName: DeskForge
```

```
directories:
```

```
  output: dist
```

```
files:
```

```
  - electron/**/*
```

```
  - src/**/*
```

```
  - package.json
```

```
win:
```

```
target: nsis
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

mac:

target: dmg

linux:

target: AppImage

Comando: `npx electron-builder --win --x64` genera el instalador.

Ejemplo: "App de Notas con Notificaciones Nativas y Atajos Globales"

Para que vean cómo funciona Electron en un caso simple, construyamos un bloc de notas con notificaciones y atajos globales.

Estructura:

```
notes-app/  
├── electron/  
│   ├── main.js  
│   └── preload.js  
├── src/  
│   ├── index.html  
│   ├── styles.css  
│   └── app.js  
└── package.json
```

`electron/main.js`:

```
const { app, BrowserWindow, ipcMain, dialog, Notification, globalShortcut } =  
require('electron');  
const path = require('path');  
const fs = require('fs');
```

```
let win;
```

```
function createWindow() {
  win = new BrowserWindow({
    width: 900,
    height: 700,
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
      contextIsolation: true
    }
  });
  win.loadFile('src/index.html');
}

app.whenReady().then(() => {
  createWindow();

  ipcMain.handle('file:open', async () => {
    const res = await dialog.showOpenDialog({ properties: ['openFile'] });
    if (res.canceled) return null;
    return { path: res.filePaths[0], content: fs.readFileSync(res.filePaths[0], 'utf-8') };
  });

  ipcMain.handle('file:save', async (_, { content }) => {
    const res = await dialog.showSaveDialog({ defaultPath: 'nota.txt' });
    if (res.canceled) return false;
    fs.writeFileSync(res.filePath, content, 'utf-8');
    new Notification({ title: 'Guardado', body: 'Tu nota se guardó correctamente' }).show();
    return true;
  });

  globalShortcut.register('CommandOrControl+Shift+N', () => {
    win.webContents.send('shortcut:newNote');
  });
});
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

```
});
```

```
app.on('will-quit', () => globalShortcut.unregisterAll());
```

`electron/preload.js`:

```
const { contextBridge, ipcRenderer } = require('electron');
```

```
contextBridge.exposeInMainWorld('api', {  
  openFile: () => ipcRenderer.invoke('file:open'),  
  saveFile: (content) => ipcRenderer.invoke('file:save', { content }),  
  onNewNote: (cb) => ipcRenderer.on('shortcut:newNote', cb)  
});
```

`src/app.js`:

```
document.getElementById('open').addEventListener('click', async () => {  
  const file = await window.api.openFile();  
  if (file) document.getElementById('editor').value = file.content;  
});
```

```
document.getElementById('save').addEventListener('click', async () => {  
  const content = document.getElementById('editor').value;  
  await window.api.saveFile(content);  
});
```

```
window.api.onNewNote(() => {  
  document.getElementById('editor').value = "";  
});
```

Este mini proyecto demuestra el flujo completo de IPC (invoke/handle y send/on), uso de APIs nativas (diálogos, notificaciones, atajos globales) y separación segura con preload. En DeskForge aplicarán los mismos patrones pero sumando audio y visualizaciones.

Proyecto DeskForge + BeatSync – Descripción y Requisitos.

Descripción General.

DeskForge + BeatSync es una suite de productividad de escritorio que combina:

- **Herramientas de productividad:** Bloc de notas con autoguardado, temporizador Pomodoro, atajos globales, bandeja del sistema.
- **Reproductor musical interactivo (BeatSync):** Playlists, visualizaciones en tiempo real, ecualizador, integración con Spotify y Last.fm.

Requisitos Funcionales (ID).

ID	Requerimiento	Descripción
E1	Ventana principal	App Electron con ventana redimensionable, menú nativo y bandeja del sistema.
E2	Bloc de notas	Editor con apertura/guardado de archivos (diálogos nativos), autoguardado local.
E3	Temporizador Pomodoro	Ciclos de 25/5 min con notificaciones nativas al finalizar.
E4	Reproductor musical	Controles play/pause/next/prev, barra de progreso, volumen, shuffle, repeat.
E5	Carga de música local	Selección de carpeta, listado de MP3/OGG/WAV, reproducción con Howler.js.
E6	Visualizador de audio	Al menos 2 tipos: barras de frecuencia y onda/círculo radial, con Canvas + Web Audio API.
E7	Playlists drag & drop	Crear, editar y reordenar playlists arrastrando canciones. Persistencia local.
E8	Integración Spotify / Last.fm	Búsqueda de canciones en Spotify (OAuth) y/o scrobbling con Last.fm.

Proyecto. Diseño e Implementación de Sitios Web Dinámicos.

E9	Atajos globales	Al menos 2 atajos (ej. Ctrl+Shift+Space para play/pause, Ctrl+Shift+P para Pomodoro).
E10	Empaquetado	Generar instalador `.exe` (Windows) o `.AppImage` (Linux) con electron-builder.
E11	Ecualizador	Sliders de bajos, medios, agudos aplicados con Web Audio API (BiquadFilterNode).
E12	Tema oscuro/claro	Detección del sistema y persistencia de preferencias.

Objetivos del Proyecto.

- **Técnico:** Dominar Electron, IPC, APIs nativas y audio en el navegador.
- **Arquitectónico:** Separar main process, preload y renderer; estructurar el proyecto para empaquetado.
- **Práctico:** Reproducir audio, analizarlo y visualizarlo en tiempo real.
- **Profesional:** Publicar un instalador descargable y consumir APIs externas reales.

Comandos clave para empezar:

```
npm init -y  
npm install --save-dev electron electron-builder vite  
npm install howler  
npm install --save-dev concurrently wait-on
```

`package.json` (fragmento):

```
{  
  "main": "electron/main.js",  
  "scripts": {  
    "dev": "concurrently \"vite\" \"wait-on http://localhost:5173 && electron .\"",  
    "build": "vite build && electron-builder"
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

}

}

Criterios de Evaluación.

Evaluación Continua (60% – Durante la clase)

- **Conexión y participación:** Estar en Live Share, interactuar, hacer preguntas.
- **Avance parcial:** Antes de finalizar la clase, subir un `.zip` del proyecto (excluyendo `node\_modules`) o enlace a repositorio GitHub/Drive con el avance. Debe incluir:
 - Proyecto Electron corriendo (ventana abierta).
 - IPC funcionando (al menos un handler y un invoke).
 - Reproductor reproduciendo al menos un archivo de audio local.
 - Primer visualizador (barras o waveform) funcionando.
- **Calidad de código:** Uso de `contextIsolation: true`, `preload.js` para exponer APIs, organización de carpetas, sin mezclar Node en el renderer.

Evaluación Final (40% – Entrega hasta el martes siguiente)

- **Funcionalidad:** Cumplir con al menos 7 de los 12 requisitos (E1 a E12).
- **Reproductor:** Debe reproducir múltiples formatos y permitir navegación entre pistas.
- **Visualizaciones:** Al menos 2 tipos distintos, fluidos y reactivos.
- **Integración externa:** Spotify o Last.fm funcionando (puede ser búsqueda simple).
- **Empaquetado:** Entregar el instalador generado con electron-builder.
- **Backend/Cloud (si aplica):** Si usan un backend para playlists o sync, desplegarlo en Vercel/Render/Netlify.
- **Informe técnico:** Documento con arquitectura, decisiones, problemas y soluciones.
- **Código limpio:** TypeScript (opcional pero recomendado), comentarios, buenas prácticas, separación de responsabilidades.

Entregables.

Durante la clase (Avance Parcial)

1. Enlace de Live Share: Compartir en el comentario privado de la plataforma **antes de empezar a codificar.

2. Avance Parcial: Subir a la plataforma:

- `.zip`` del proyecto (sin `node_modules`` ni `dist``), O
- Enlace a repositorio GitHub con el avance, O
- Enlace a Google Drive con el archivo comprimido.

3. Capturas o GIF mostrando la ventana, el reproductor y el visualizador funcionando.

Entrega Final (Martes siguiente)

1. Repositorio GitHub: Repositorio público con el código completo y `README.md`` bien documentado.

2. Instalador: Archivo `.exe`` (Windows), `.dmg`` (macOS) o `.AppImage`` (Linux) generado con electron-builder. Subir a GitHub Releases o Drive.

3. Backend desplegado: Si aplica (sync de playlists, API propia), URL en Vercel/Render/Netlify.

4. Informe Técnico: En PDF o Markdown, que incluya:

- Introducción y objetivos.
- Explicación de la arquitectura (main, preload, renderer, IPC).
- Capturas de pantalla de cada funcionalidad.
- Capturas o GIF de las visualizaciones de audio.
- Dificultades encontradas y soluciones.
- Conclusiones y aprendizajes.
- Enlace al instalador y al backend desplegado (si aplica).

5. Video demo: Grabación de 2-3 minutos mostrando la app corriendo.

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

Deben subir ambas versiones: el avance parcial de la clase y la versión final.

Sistema Integrador y Defensa Final.

- **Recordatorio clave:** DeskForge + BeatSync no es un proyecto aislado. Forma parte del sistema integrador del segundo cuatrimestre, donde deberán combinar todas las tecnologías vistas durante el año (frontend web, frameworks mobile, backend, bases de datos, cloud, desktop) en un proyecto unificado, o bien acoplarlo a su proyecto final.
- **Defensa final:** Al final del cuatrimestre, defenderán su proyecto final explicando:

- La arquitectura y decisiones técnicas.
- Cómo se integran las diferentes tecnologías (web + mobile + backend + desktop).
- El código (cada línea importante).
- Los desafíos y cómo los resolvieron.

- **Enfoque en código:**

- Usar ``contextIsolation: true`` y ``nodeIntegration: false`` (seguridad en Electron).
- Separar la lógica de negocio de la UI.
- Comentar partes complejas (especialmente IPC y Web Audio).
- Mantener un historial de commits claro y descriptivo.
- Escribir código modular, reutilizable y testeable.
- Documentar con JSDoc las funciones exportadas.